

Voice-Driven Transfer Function & Viewpoint Control

Architecture Reference — Current State

Kevin Kunkel

Universität Leipzig · Masterseminar: Optimierung von Transferfunktionen
im Volume Rendering mit Reinforcement Learning

Betreuer: Prof. Gerik Scheuermann, Dr. Thomas Wischgoll, Dr. Baldwin Nsonga

September 2026

This document describes the system as it stands today: a local prototype that turns spoken or typed commands into transfer-function and camera edits on volume-rendered CT data, plus two independent reinforcement-learning sub-projects that each learn to replace a hand-tuned hill-climbing baseline. It is a technical reference for the codebase, not a thesis chapter — for the narrative/motivation framing see `slides/slides.typ`, and for a boxes-and- arrows diagram of the same system see `architecture-mvp.drawio` in the repo root.

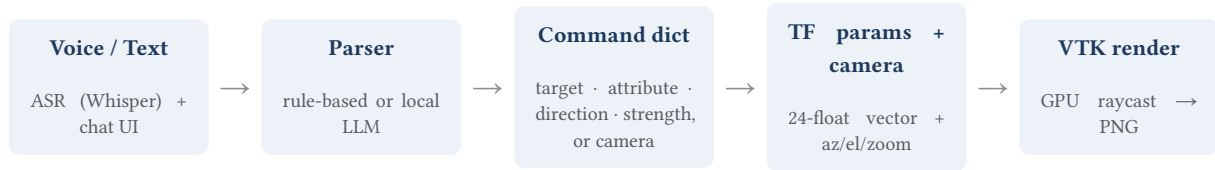
Contents

1. Overview	3
1.1. Module map	3
2. Rendering pipeline	5
2.1. Data sources	5
2.2. Transfer function representation	5
2.3. VTK render	5
3. Command layer	7
3.1. Rule parser	7
3.2. LLM parser	7
3.3. Applying a command	7
4. Search and evaluation	9
4.1. Step math (<code>search.py</code>)	9
4.2. Evaluators (<code>evaluate.py</code>)	9
5. Camera control	10
5.1. Why camera state lives on the backend	10
6. RL sub-project 1 — transfer-function opacity	11
6.1. Environment (<code>rl/env.py</code>)	11
6.2. Training and evaluation	11
6.3. Result (200k timesteps, 20 held-out episodes)	11
7. RL sub-project 2 — camera viewpoint	12
7.1. Why a real CT dataset, not the synthetic phantom	12
7.2. Environment (<code>rl/camera_env.py</code>)	12
7.3. Training and evaluation	12
7.4. Result (200k timesteps, <code>ct_skull</code> , 20 held-out episodes)	13
8. Server, session model, and UI	14
8.1. Session state	14
8.2. Persistence	14
8.3. UI	14

9. Known limitations and next steps	15
9.1. Roadmap toward voice-driven camera control in browser and VR	15

1. Overview

The system has five layers, each independently swappable without touching its neighbors:



A rendered step feeds an evaluator (objective metric or human judgment), which drives a hill-climbing search loop back into the command layer — see Section 4.

Two Gymnasium environments mirror pieces of this pipeline for offline RL training, without touching VTK rendering at all: one trains against the transfer-function opacity metric directly (Section 6), the other against a camera-viewpoint alignment proxy computed straight from the volume’s voxel data (Section 7). Both are trained, evaluated, and compared against the hand-coded hill-climbing baseline they are meant to eventually replace — neither is wired into the live chat/voice loop yet.

1.1. Module map

File	Responsibility
<code>phantom.py</code>	Synthetic CT volume in Hounsfield units (fallback dataset)
<code>datasets.py</code>	Real CT dataset registry: checksum-verified download, NRRD loading, caching
<code>transfer.py</code>	24-float TF vector ↔ VTK transfer functions, <code>opacity_mass</code> / <code>mass_fraction</code> metrics
<code>render.py</code>	Offscreen VTK volume render, pixel grab, image features
<code>camera.py</code>	Camera state (azimuth/elevation/zoom) and relative camera commands
<code>commands.py</code>	Rule-based parser, local-LLM parser, <code>apply_command</code> , <code>COMMAND_REFERENCE</code>
<code>search.py</code>	Hill-climbing step math (<code>propose_step</code> , <code>resize_step</code>), shared across every consumer
<code>evaluate.py</code>	Objective (<code>opacity_mass</code> -based) and human (console) evaluators, preference logging
<code>asr.py</code>	faster-whisper push-to-talk mic capture and file transcription
<code>mvp.py</code>	CLI entry point: single apply, hill-climb loop, state persistence

File	Responsibility
server.py + static/	FastAPI chat UI: text/voice commands, history navigation, human judging
rl/env.py, rl/train.py, rl/eval.py	RL sub-project 1: transfer-function opacity (SAC)
rl/camera_env.py, rl/camera_train.py, rl/camera_eval.py	RL sub-project 2: camera view-point (SAC)
plots/	Thesis figure generation from SB3 training logs

2. Rendering pipeline

2.1. Data sources

Every command ultimately acts on a 3D numpy array of Hounsfield Units (HU) plus a physical voxel spacing. Two sources exist behind one interface, `datasets.load_dataset(name) -> (volume, spacing)`:

- **synthetic** — `phantom.build_phantom()`: a fixed-seed, layered blob phantom (torso $\rightarrow$ fat $\rightarrow$ spongy $\rightarrow$ cortical bone, each a smooth sigmoid-edge blob composited additively, plus Gaussian noise). Fast, deterministic, no network access, but its outer tissues (air, fat, soft) are geometrically symmetric around the volume center — a limitation that mattered for camera-viewpoint RL (Section 7).
- **Four real CT datasets** (`ct_chest`, `ct_skull`, `ct_cardio`, `ct_abdomen`) — de-identified public test data from the 3D Slicer project, each downloaded once from a content-addressed GitHub release URL, SHA-256 checksum-verified against Slicer’s own registration, and cached under `data/`. Loaded via `vtkNrrdReader`, with an explicit axis transpose so the resulting numpy array’s axis 0 matches VTK’s X axis exactly as `render.py` expects (`ravel(order="F")`).

2.2. Transfer function representation

The transfer function is a flat 24-float vector: 4 Gaussian peaks $\times$ 6 parameters each (center, width, height, R, G, B), always stored normalized to $[-1, 1]$ and mapped to real units on demand:

```
N_PEAKS = 4
PARAMS_PER_PEAK = 6 # center, width, height, r, g, b
CENTER_RANGE = (-1050.0, 2000.0) # HU
WIDTH_RANGE = (10.0, 400.0) # HU
```

Five tissues are recognized throughout the system by fixed HU bands, used identically by the transfer function, the command layer, and both RL sub-projects:

Tissue	Representative HU	Band (HU)
air	-1000	-1050 to -550
fat	-100	-550 to -30
soft	40	-30 to 170
spongy (cancellous bone)	300	170 to 600
bone (cortical)	900	600 to 2000

At render time, `transfer.vector_to_vtk` samples 256 HU points, evaluates each peak’s Gaussian, composites total opacity (clipped to $[0, 1]$) and a weight-blended RGB, and builds a `vtkColorTransferFunction + vtkPiecewiseFunction` pair. Two derived metrics matter everywhere else in the codebase:

- **opacity_mass(params, lo, hi)** — the exact trapezoidal integral of composited opacity over an HU sub-range. This is the ground-truth signal the hand-coded evaluator and both RL rewards are built on; no rendering is needed to compute it.
- **mass_fraction(params, tissue)** — `opacity_mass` normalized by the band’s width, giving an average-opacity-in-band fraction in $[0, 1]$. Raw `opacity_mass` scales with band width (bone’s band alone is 1400 HU wide), which would bias any learner toward wide bands; `mass_fraction` removes that bias and is what the TF-opacity RL environment observes and rewards.

2.3. VTK render

`render.render(volume, params, spacing, camera)` builds a `vtkImageData` from the HU array, attaches the transfer function via a `vtkVolumeProperty`, and picks a GPU raycast mapper with an automatic CPU fallback (`vtkFixedPointVolumeRayCastMapper`) when the GPU path isn’t supported. The camera is applied in a fixed order that matters: `ResetCamera()` first (so the volume fills the frame), then relative

`Azimuth()` / `Elevation()` / `Zoom()` calls, then `ResetCameraClippingRange()` — reusing the same relative-adjustment shape the camera command layer already uses (Section 5). `render.grab()` reads back an RGB numpy array from the offscreen window; `render.features()` computes cheap image statistics (mean, std, coverage, entropy) that get logged alongside every command for later analysis, independent of any evaluator.

3. Command layer

Every user-facing instruction becomes one command dict before anything else happens. There are five shapes:

```
{ "target": "bone", "attribute": "opacity", "direction": "increase", "strength":  
  "strongly" }  
{ "target": "bone", "attribute": "opacity", "direction": "set", "level": "high" }  
# absolute  
{ "compound": [ { ... }, { ... } ] }  
# multi-tissue absolute  
{ "target": [ "bone", "spongy" ], "attribute": "opacity", "direction": "show_only",  
  "strength": None }  
{ "camera": { "action": "rotate", "direction": "right", "strength": "moderately" } }  
# no TF target at all
```

COMMAND_REFERENCE (in `commands.py`) is the single source of truth for what the system understands — it drives the rule parser’s regex grammar, the LLM’s system prompt, **and** the in-UI help dialog, so the three can never silently drift apart:

Category	Example
Opacity (relative)	increase opacity for bone strongly
Opacity (absolute)	high opacity spongy
Show only	show only bone and spongy
Compound	high opacity spongy, low opacity bones
Width / sharpness	sharpen the bone peak
Brightness	brighten bone
Center position	shift bone's center up
Camera	rotate left, tilt down, zoom in
Reset	reset

3.1. Rule parser

`parse_command_rule` runs an ordered cascade of regexes (reset → show-only → sharpen/soften/brighten/darken → shift-center → camera rotate/tilt/zoom → absolute level(s) → relative increase/decrease), raising `ValueError` if nothing matches. Tissue names are resolved through a synonym table (TISSUE_SYNONYMS) sorted longest-phrase-first, so "cortical bone" matches before the bare "bone" it contains.

3.2. LLM parser

`parse_command_llm` sends the raw text plus a strict-JSON system prompt to a local Ollama server (default model `qwen2.5:7b`), enforcing `format: "json"`. The prompt encodes the tissue synonym table (generated from the same TISSUE_SYNONYMS the rule parser uses), the full command schema including the separate camera-command shape, and a rule for direction that follows the user’s **goal** rather than their wording ("barely there" → increase, because something invisible needs more presence). Any failure — network error, malformed JSON, or a schema the validator rejects — falls back to the rule parser and logs the full request/response/fallback-reason to `out/llm_requests.jsonl` for later accuracy analysis (`eval_parsers.py`).

3.3. Applying a command

`apply_command` resolves a target tissue to a concrete peak via `_find_or_create_peak` (nearest existing peak by HU distance, or reseed the **weakest** peak if nothing is within 300 HU), then mutates the relevant parameter slot(s):

```
ATTRIBUTE_PARAM_INDEX = {"opacity": 2, "width": 1, "brightness": (3, 4, 5),  
"center": 0}
```

Two details prevent the naive version of this from misbehaving:

- **_asymptotic_step** moves a fraction of the **remaining headroom** toward the ± 1 bound, not a fixed absolute amount — a fixed step would saturate almost any starting value in one command. This generalizes a fix originally found for height/opacity to every attribute stored in the same normalized encoding.
- **_center_band_clip** keeps a center-shift command inside the target tissue's own HU band, so repeated "shift bone's center up" commands can't silently walk the bone peak into cortical-adjacent territory outside its own tissue's meaning.

4. Search and evaluation

Hill-climbing is the system’s baseline optimizer, used directly by the CLI and chat UI, and later used as the comparison point for both RL sub-projects.

4.1. Step math (`search.py`)

```
def propose_step(params, peak_idx, sign, step):
    ... # mutates only the resolved peak's height slot

def resize_step(step, accepted, max_step=1.0):
    return min(step * 1.2, max_step) if accepted else step * 0.5
```

`propose_step` is opacity-only by convention — callers must gate entry on `attribute == "opacity"`, since it always mutates the height slot regardless of what a command dict says. `resize_step` is domain-agnostic: it grows the step 1.2× on acceptance (capped at `max_step`) and halves it on rejection. The `max_step` parameter was added during the camera-viewpoint RL sub-project after a real bug: the original hardcoded 1.0 cap was tuned for the opacity domain (values live in $[0, 1]$) and silently capped the camera hill-climb’s step at 1° instead of a 30°-scale exploration budget — see Section 7 for the numbers this cost.

4.2. Evaluators (`evaluate.py`)

`objective(params_before, params_after, cmd)` returns +1/−1:

- Camera commands, compound/reset commands, and width/brightness/center commands are always accepted — there is no established exact metric for them the way `opacity_mass` exists for opacity, and (for camera/compound) no well-defined “wrong direction” either.
- `show_only` is judged by **dominance share** (target tissue’s fraction of total opacity mass across all bands), not raw mass — crushing every other peak can lower the target’s own mass while still making it far more dominant in the image.
- A relative increase/decrease is accepted only if the target band’s `opacity_mass` moved the right way **and** no other tissue’s mass moved by more than the target did (a simple collateral-damage check).

`human(before_png, after_png)` is a console b/s prompt; the chat UI exposes the same judgment as Better/Worse buttons over a live-rendered image pair (Section 8).

5. Camera control

The camera is modeled as its own small piece of state, deliberately kept independent of the transfer-function vector:

```
DEFAULT_CAMERA = {"azimuth": 30.0, "elevation": 20.0, "zoom": 1.0}
ELEVATION_RANGE = (-85.0, 85.0)
ZOOM_RANGE = (0.3, 4.0)
ROTATE_STEP_DEGREES = {"slightly": 15.0, "moderately": 30.0, "strongly": 60.0}
ZOOM_STEP_FACTOR = {"slightly": 1.15, "moderately": 1.35, "strongly": 1.7}
```

`apply_camera_command` applies one relative adjustment: azimuth **wraps** modulo 360° (it has no natural bound), while elevation and zoom **clamp** to fixed ranges (both have real physical limits — clamping elevation specifically avoids the camera flipping through the poles). Camera commands are a fully separate shape from transfer-function commands at every layer: their own regex branch in the rule parser, their own schema block in the LLM system prompt, their own `_validate_camera_cmd` check, and (critically) they never touch the 24-float TF vector at all.

5.1. Why camera state lives on the backend

In the chat UI, camera state is stored **per history step** (`Session.history[i]["camera"]`), not as client-side view state. A camera-only command creates a new history step that carries the same transfer-function params forward but a new camera; back/forward navigation restores both together. This was a deliberate design choice, not an accident: it makes camera state part of the same server-owned session model as everything else, which is the actual prerequisite for a **different** client — a browser tab today, potentially a native Vrui/CAVE application later — to drive or observe the same session without duplicating logic (see Section 9).

One real bug surfaced here during review: `Session.judge()`'s convergence branch originally sourced the **final** camera from live cursor state instead of the camera frozen at the moment a human judgment began. If a user rotated the camera while a judgment was pending, the eventual accepted step could silently end up with the wrong camera. The fix freezes the camera into the pending-judgment dict itself (`p["camera"]`) and reads it back from there, so a mid-judgment camera change updates the **live** cursor (as expected) without corrupting the judged step.

6. RL sub-project 1 — transfer-function opacity

The first RL sub-project asks: can a learned policy match or beat the hand-coded hill-climber at the task it already does — nudging one tissue’s opacity up or down? Training happens entirely offline against the exact `opacity_mass` metric; no rendering, no human data.

6.1. Environment (`rl/env.py`)

Aspect	Definition
Episode	One (<code>target_tissue</code> , <code>direction</code>) goal, sampled fresh on <code>reset()</code> , from a randomly height-perturbed default transfer function
Observation	31-dim: the 24 TF params + 5-way tissue one-hot + direction flag + current <code>mass_fraction</code>
Action	1-dim continuous delta in $[-1, 1]$, scaled by <code>MAX_DELTA = 0.2</code> , applied to the resolved target peak’s height via the same <code>search.propose_step</code> the hill-climber uses
Reward	Signed per-step Δ <code>mass_fraction</code> in the target band (positive if it moved the right way)
Episode length	20 steps, truncated (not terminated)

Reusing `commands._find_or_create_peak` and `search.propose_step` means the RL agent moves the transfer function through **exactly** the same mechanism as a real user command — it cannot exploit some RL-only shortcut unavailable to the baseline.

6.2. Training and evaluation

`rl/train.py` trains a `stable-baselines3` SAC agent (`MlpPolicy`, 4 parallel environments via `make_vec_env`) for a configurable timestep budget, logging per-seed CSV + TensorBoard output. `rl/eval.py` then replays the **same** held-out (`params`, `target_tissue`, `direction`) episodes through both the trained policy (deterministic) and the hill-climbing baseline (`propose_step` + `resize_step`, gated by `objective()`), reporting mean final `mass_fraction` and mean steps-to-90%-of-best (`_steps_to_90pct`) — a generic helper reused unmodified by the second RL sub-project.

6.3. Result (200k timesteps, 20 held-out episodes)

	Mean final <code>mass_fraction</code>	Mean steps to 90% of best
Trained SAC policy	0.344	3.3
Hill-climb baseline	0.347	2.7

Interpretation: Near-parity, with the hand-tuned hill-climber converging slightly faster. This is a legitimate finding, not a failed experiment: a well-tuned hand-designed controller is a strong baseline on a low-dimensional, single-scalar-action problem like this one. The value of the RL approach is expected to grow once the action space widens (full multi-peak control rather than one resolved peak’s height) or the reward incorporates signal a hand rule can’t easily express (e.g. learned human preference). The agent is trained and evaluated, but **not** wired into the live chat/voice loop.

7. RL sub-project 2 — camera viewpoint

The second RL sub-project takes inspiration from natural-language viewpoint-navigation work (literature/ZhaoTao2025_NL-Viewpoint-Navigation-RL.pdf) but scopes down sharply: instead of language-conditioned navigation against a learned reward, it trains a continuous-control agent to orient the camera toward a target tissue’s centroid direction, against a cheap, exact geometric proxy computed directly from voxel data — no rendering, no learned reward model.

7.1. Why a real CT dataset, not the synthetic phantom

The phantom’s outer tissues (air, fat, soft) were assumed, from reading `phantom.py`’s blob-center coordinates, to be spatially symmetric around the volume center. Direct measurement during design contradicted this: `soft` tissue’s HU-band mask inherits an off-center bias from the bone/spongy blobs’ smooth sigmoid-edge transitions overlapping into its own HU range, so `soft` actually clusters with spongy/bone (approx. 5% relative centroid offset) rather than with `air`/fat (approx. 1%). Since a genuinely-centered tissue has no meaningful “direction” to align a camera toward, the phantom does not reliably exercise this environment across all five tissues — the design was changed to train on a real CT scan (`ct_skull`) instead, where anatomical asymmetry is real rather than a modeling artifact.

7.2. Environment (`rl/camera_env.py`)

Aspect	Definition
Episode	One target tissue, resolved on <code>reset()</code> from a random starting azimuth/elevation
Observation	9-dim: sin/cos of azimuth, normalized elevation, 5-way tissue one-hot, current alignment
Action	2-dim continuous ($\Delta\text{azimuth}$, $\Delta\text{elevation}$) in $[-1, 1]$, scaled by <code>MAX_DELTA_DEGREES = 30°</code>
Reward	Per-step increase in alignment — the dot product of the camera’s view direction and the target tissue’s centroid direction from the volume center
Episode length	20 steps, truncated (not terminated)

A target tissue’s centroid direction is computed once per episode:

```
def _tissue_centroid_direction(volume, spacing, tissue):
    mask = (volume >= lo) & (volume < hi)          # tissue's HU band
    centroid = voxel_mean(mask, spacing)           # physical-space centroid
    offset = centroid - volume_center
    if norm(offset) < MIN_RELATIVE_CENTROID_OFFSET * volume_diagonal:
        return None # too close to center to have a meaningful direction
    return offset / norm(offset)
```

`_resolve_target` shuffles the five tissues and picks the first with a meaningful direction, falling back to bone — and raises a clear `ValueError` (rather than silently propagating `None` into a downstream `TypeError`) if even that fallback has no valid direction. This was itself a bug found and fixed during implementation: the original fallback could return `("bone", None)`.

7.3. Training and evaluation

`rl/camera_train.py` mirrors `rl/train.py`’s structure exactly (same SAC/SB3 recipe, same per-seed logging convention, disjoint model/log paths so the two agents never collide on disk), parameterized by which real dataset to load. `rl/camera_eval.py` compares the trained policy against a 2D coordinate hill-climbing baseline that probes four cardinal azimuth/elevation directions per step, reusing `search.resize_step` and `rl.eval._steps_to_90pct` directly rather than reimplementing either.

A real bug caught during this sub-project: The hill-climb baseline seeds its step at `MAX_DELTA_DEGREES` (30°), but `search.resize_step`’s growth cap was hardcoded to 1.0 — correct for every **other** caller, all of which operate on an opacity parameter in $[0, 1]$. The very first accepted move collapsed the camera baseline’s step from 30° to 1°, and it could never grow back. A smoke-test run with the bug present reached only **0.658** mean final alignment; after adding an optional `max_step` parameter (defaulting to 1.0, so the three pre-existing opacity-scale callers are completely unaffected) and passing `max_step=MAX_DELTA_DEGREES` at the camera call site, the same setup reached **0.99998**. A regression test now locks in the correct wiring at that call site specifically, since none of the pre-existing tests would have caught a future regression of it.

7.4. Result (200k timesteps, `ct_skull`, 20 held-out episodes)

	Mean final alignment	Mean steps to 90% of best
Trained SAC policy	0.997	3.05
Hill-climb baseline	1.000	4.05

Interpretation: Both methods reach near-perfect alignment — expected, since the alignment landscape here is smooth and unimodal (a dot product against a fixed target direction), which a well-calibrated hill climb can solve almost exactly. The trained policy’s advantage is convergence **speed**: it reaches 90% of its achievable alignment about one step sooner on average, which is the more practically relevant property for an interactive tool where every step costs a render.

Known limitation: The centroid-alignment reward is an intentional, acknowledged proxy for “can you actually see this tissue” — it has **no occlusion model**. A camera can achieve high alignment while another structure blocks the actual view. Ray-marching-based visibility scoring was considered and explicitly deferred as a separate, much larger effort. The action space is also angle-only (azimuth/elevation) — zoom is not controlled — by deliberate scoping decision.

8. Server, session model, and UI

`server.py` exposes a FastAPI app whose route handlers are thin `async def` wrappers around a plain-Python `Session` class with no FastAPI or threading dependency — kept independently unit-testable, and kept `async` specifically so Starlette runs handlers on the main event-loop thread rather than a worker pool (VTK’s Cocoa render window can only be created on the true process main thread on macOS).

8.1. Session state

```
class Session:
    history: list[dict] # one entry per applied step
    cursor: int         # current position for back/forward navigation
    pending: dict | None # an in-progress human-judgment pair, if any
```

Each history entry carries its transfer-function params, its camera dict, computed tissue masses, image features, the command that produced it, and any thumbs-up/down feedback — camera and TF state travel together but are never conflated (Section 5). A new command truncates any existing forward history from the current cursor, matching ordinary browser back/forward semantics. A pending human-judgment entry freezes a before/after image pair and the camera in effect at the time, so the judgment being evaluated can never be silently corrupted by state that changes while the human is still deciding.

8.2. Persistence

Path	Contents
<code>out/ui_session.json</code>	Full session (history + cursor), reloaded across server restarts
<code>out/ui_images/<session_id>/</code>	Every rendered PNG, namespaced per session so ids never collide across dataset switches
<code>out/log.jsonl</code>	Every applied step and hill-climb iteration, with before/after params and verdict
<code>out/preferences.jsonl</code>	Human judgments with before/after image paths and the objective evaluator’s agreeing/disagreeing verdict
<code>out/feedback.jsonl</code>	Thumbs up/down ratings on any rendered step
<code>out/llm_requests.jsonl</code>	Every LLM-parser call: raw response, parsed command, whether/why it fell back

8.3. UI

The static frontend (`static/index.html`, `app.js`, `style.css`) is a chat-style interface: type or speak a command (🗣️, via the browser mic and the same `faster-whisper` wrapper used by the CLI), watch it render, step back/forward through history, judge pending hill-climb pairs as Better/Worse, leave thumbs-up/down feedback on any step, switch datasets, and open a command-reference help dialog generated from the exact same `COMMAND_REFERENCE` table the parsers use internally.

9. Known limitations and next steps

- **Neither RL agent is live.** Both sub-projects are trained and evaluated entirely offline against the same metrics the hand-coded hill-climber already uses; neither is currently reachable from the chat/voice loop.
- **Camera-viewpoint reward has no occlusion model** (Section 7) — it is a directional-alignment proxy, not a rendering-accurate visibility claim.
- **Camera-viewpoint RL controls angle only**, not zoom — an approved scoping decision.
- **Everything runs locally.** Ollama, faster-whisper, and VTK's offscreen renderer all run on-machine; no data or audio leaves the process.

9.1. Roadmap toward voice-driven camera control in browser and VR

The longer-term goal is a camera the user can drive by voice, with the same backend logic reachable from both a browser client and, eventually, a native client for a lab VR rig (Vrui/CAVE-based). That was decomposed into four sub-projects:

1. **Camera as controllable state** — done. Camera state lives in the backend session, addressed relatively (rotate/tilt/zoom deltas) exactly like transfer-function commands are.
2. **RL for camera-viewpoint optimization** — done (Section 7).
3. **Client/backend split** — satisfied by construction: because camera state already lives server-side rather than in any one client's view state (Section 5), a second client does not require re-deriving this logic.
4. **A native Vrui/CAVE client** reusing the same backend Session/command logic — deliberately deferred until an upcoming lab visit clarifies the actual VR hardware available, since Vrui targets specific CAVE/lab rig configurations rather than being a general browser-compatible toolkit the way WebXR is.